

과제 8 · 제출 보고서

내 소개 페이지에 패스키 달기 — 비밀번호 없이 나만 들어가기

공개 소개는 그대로 두고, 새로 만든 “나만 보는 자리”만 패스키로 잠갔습니다.
막았다는 문장 대신 **막히는 장면**(성공한 요청과 거절된 요청)으로 보입니다.

결과물 주소	https://aleph-passkey.vercel.app
소스 주소	https://github.com/ginye28/ALEPH/tree/main/portfolio-passkey
불인 방식	WebAuthn (FIDO2 패스키) · 비밀번호 없음
검사	42개 통과 / 0개 미통과 (총 42개)
증빙 촬영	사진 10장 · 기록 16건
작성일	8 september 2026

짧은 확인 방법

어디로 가나요

<https://aleph-passkey.vercel.app> — 첫 화면은 누구나 볼 수 있는 공개 소개 페이지입니다. 계정을 만들 필요도, 로그인할 필요도 없습니다.

세 단계 안에 무엇을 하나요

- ① 페이지 맨 아래  **나만 보는 자리**에서 “패스키 새로 등록하기”를 눌러 기기의 지문·얼굴·PIN으로 등록합니다.
- ② 열린 자리에 메모를 한 줄 남깁니다. ③ “로그아웃”을 누른 뒤 같은 자리를 다시 봅니다.

무엇이 보이면 통과인가요

①·② 뒤에는 잠금 배지가 **열림**으로 바뀌고 비공개 항목이 보입니다. ③ 뒤에는 다시 **잠김**으로 돌아가고 내용이 사라집니다. 이때 브라우저에서 페이지 소스를 열어 봐도 비공개 내용은 없습니다.

안 될 때는 무엇이 보이나요

패스키를 지원하지 않는 브라우저면 “이 브라우저는 패스키를 지원하지 않습니다”라는 안내가 뜨고 버튼이 꺼집니다. 등록 도중 지문 창을 닫으면 “등록을 취소했습니다. 서버에는 계정도 패스키도 만들어지지 않았습니다”가 뜹니다. 기기에 패스키가 하나도 없는 상태에서 “패스키로 들어가기”를 누르면 브라우저가 고를 것이 없다고 알려 줍니다.

인증 구현 설명서 — ① 무엇으로 붙였나

WebAuthn(FIDO2) 표준을 씁니다. 비밀번호는 **아예 만들지 않았**습니다. 직접 구현한 것이 아니라 아래 조각들을 골라 썼습니다.

자리	쓴 것	무엇을 맡겼나
브라우저	<code>navigator.credentials.create()</code> / <code>.get()</code> (브라우저·OS 내장)	키 쌍 생성, 개인키 보관, 지문·얼굴·PIN 확인, 서명
서버 검증	@simplewebauthn/server 13.3.3 (MIT)	질문(challenge) 생성, 서명·origin·rpID 검증, 서명 유효성 확인
서버 실행	Vercel 서버리스 함수 (Node)	api/ 폴더의 파일 하나가 주소 하나
저장소	Supabase Postgres (service_role, 서버에서만)	공개키·질문·세션·비공개 자료
브라우저 쪽 연결 코드	직접 작성 (app.js 약 40줄)	base64url ↔ ArrayBuffer 형식 변환만. 보안 판단은 한 줄도 없음

브라우저 쪽 40줄을 직접 쓴 이유 — 하는 일이 **자료 형식 변환**뿐이라 라이브러리를 하나 더 끌어올 만큼의 일이 아니었습니다. 반대로 **서명이 맞는지 판단**하는 쪽은 실수가 곧 구멍이 되는 자리라 검증된 표준 라이브러리에 그대로 맡겼습니다.

② 왜 그걸 골랐나

고른 이유는 ①에 적었습니다. 여기에는 **검토했지만 고르지 않은 것**과 그 이유를 나란히 둡니다 — 무엇을 골랐는지도 다 무엇을 버렸는지가 이유를 더 분명히 보여 주기 때문입니다.

후보	고르지 않은 이유
CBOR·COSE 파싱을 손으로 구현	WebAuthn 규격이 이미 재생 공격 방지·origin 묵기·서명 확인을 다 정의해 두었습니다. 이 과제의 배점은 표준을 이해하고 잘 고르는 것이지 바이트 파서를 다시 만드는 것이 아닙니다.
Firebase Auth · Auth0 등 SaaS의 패스키 기능	질문 생성과 검증이 통째로 남의 상자 안으로 들어가, 카드 2:3이 요구하는 “서버가 질문을 어떻게 만들고 어떻게 확인하는지”를 설명할 수 없게 됩니다.
아이디·비밀번호를 예비 수단으로 남기기	비밀번호를 하나라도 남기면 “비밀번호를 쓰지 않는다”가 사실이 아니게 됩니다. 잠금이 가장 약한 곳의 세기로 정해지기 때문입니다.
JWT 같은 서명된 토큰으로 세션 만들기	서명만으로 판단하면 로그아웃해도 만료 전까지 그 토큰이 살아 있습니다. 서버가 세션 행을 직접 들고 있으면 로그아웃이 곧 삭제라 그 즉시 끊깁니다.

③ 어디를 어떻게 고쳤나

비밀번호를 어떻게 맡아 두나

맡아 두지 않습니다. 비밀번호가 처음부터 없습니다. 화면 어디에도 비밀번호를 입력하는 칸이 없고, 소스 전체에도 `type="password"`가 없습니다. **PASS · 검사 8 (T08-C35)**

대신 서버가 갖고 있는 것은 **공개키** 하나입니다. 등록할 때 기기가 열쇠 한 쌍을 만들어 공개키만 보내고, 서명을 만드는 개인키는 기기 밖으로 나오지 않습니다.

	기록된 값
패스키 이름	[촬영] 노트북 지문
credential id	YAb89YWejdWm...
공개키	pAEBAYcgBiFYID8nodpB6jC60q-p5FA-ZPhqunsw3MwFCPWFMU71hvwq
이 값의 정체	COSE 형식의 공개키. 서명을 확인할 수는 있지만 만들어 낼 수는 없다 – 비밀번호도, 비밀번호의 해시도 아니다.
패스키가 저장된 곳	이 촬영은 크롬의 가상 authenticator(소프트웨어 기기)를 썼다. 실제 사용에서는 기기 자체(원도우 Hello·터치 ID) 또는 구글 비밀번호 관리자·보안 키에 저장된다 (T08-C26).

PASS · 검사 13 (T08-C21) PASS · 검사 14 (T08-C22)

이 값이 비밀번호가 아니라는 확인 — 저장된 문자열을 COSE 형식으로 해독하면 키 종류와 서명 알고리즘이 그대로 나옵니다(검사 14가 매번 해독해 확인합니다). **공개키로는 서명을 확인만 할 수 있고 만들어 낼 수 없습니다.** 비밀번호나 그 해시였다면 애초에 이 형식으로 해독되지 않습니다.

 나만 보는 자리

열림

위쪽 소개는 누구나 볼 수 있습니다. 여기서부터는 패스키를 등록한 본인만 열 수 있습니다. 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 공개키만 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

프로젝트 메모

준비 중인 프로젝트 메모

학습 기록을 주 단위로 묶어 보여주는 작은 도구를 구상 중. 지금은 화면 스케치만 있고 아직 코드는 없다. 공개하기엔 이르다.

지원 목록

지원하려는 곳 목록

가상의 후보 세 곳 — 세로소프트(백엔드), 아르카 데이터(플랫폼), 미르랩(사내 도구). 이력서 초안은 다음 달까지 정리 예정.

회고

스스로 쓰는 회고

이번 주에 배운 것: 화면에서 숨기는 것과 서버가 거절하는 것은 전혀 다르다는 것. 다음 주에 고칠 것: 막히는 장면을 먼저 만들고 나서 기능을 붙이기.

한 줄 더 남기기

회고

제목

내용 — 실제 개인정보는 넣지 않습니다

저장

등록된 패스키 1개

패스키는 기기에 매어 있습니다. 기기를 잃어버렸을 때를 대비해 두 개 이상 등록해 두세요 — 하나를 지워도 남은 것으로 들어올 수 있습니다.

⚠ 패스키가 하나뿐입니다. 이 기기를 잃어버리면 이 계정에 다시 들어올 방법이 없습니다 — 비밀번호도 이메일도 없어서 되살려 드릴 수단이 없습니다. 다른 기기에서 패스키를 하나 더 등록해 두세요.

[촬영] 노트북 지문

2026. 9. 8. 등록 · YAb89YWejdWm...

지우기

▼ 서버가 가진 값 보기

서버에 저장된 것은 공개키뿐입니다. 이 값으로는 서명을 확인만 할 수 있고 만들어 낼 수는 없습니다 — 비밀번호가 아닙니다. 서명을 만드는 개인키는 기기 안에만 있고 여기로 온 적이 없습니다.

pAEBAYcgBiFYID8nodpB6jC60q-p5FA-ZPhqunsw3MwFCPWFMU71hvvq

서명 횟수: 1

패스키 추가

로그아웃

화면에서도 서버가 가진 값을 그대로 펼쳐 보여준다 — 공개키뿐이다

	기록된 값
요청 본문의 필드	id, rawId, type, clientExtensionResults, response, authenticatorAttachment

	기록된 값
response 안의 필드	clientDataJSON, attestationObject, transports
설명	개인키를 담는 자리가 아예 없다 – WebAuthn 규격에 그런 필드가 존재하지 않는다. 개인키는 기기(authenticator) 안에서 만들어져 그 안에만 남는다.
응답 상태	200

PASS · 검사 15 (T08-C23)

실제 기기로 확인한 저장 위치 (T08-C26, 2026-09-03) — 위 캡처는 자동 검사용 가상 authenticator로 만든 것이고, 실제 사람이 실제 기기로 등록하면 어디에 남는지는 따로 확인했다. 공개 주소(aleph-passkey.vercel.app)에서 아이폰 Face ID로 직접 패스키를 등록하자 **Apple 자체 저장소(iCloud 키체인)**에 저장됐다 — 같은 Apple 계정을 쓰는 다른 기기에서도 이 패스키로 로그인할 수 있다는 뜻이다. 등록된 뒤 같은 계정에 두 번째 패스키를 추가하려 하자 브라우저가 "이 기기에는 이미 패스키가 등록되어 있습니다"로 거절했는데, iCloud 키체인이 기기가 달라도 같은 패스키를 동기화해 쓰기 때문이다.

들어온 사람을 어떻게 기억하나

	기록된 값
방식	서버가 들고 있는 세션 (JWT 같은 토큰이 아니다)
쿠키	pk_session=..... (값은 가림)
속성	HttpOnly=true, Secure=true, SameSite=Lax, 만료 12시간
값 안에 담긴 정보	없음 — 그저 pk_sessions 테이블의 행 하나를 가리키는 무작위 id다
페이지에서 읽히는가	아니오. document.cookie = "" (HttpOnly라 스크립트가 못 읽는다)

PASS · 검사 23 (T08-C32) PASS · 검사 24 (T08-C34)

세션입니다. 토큰이 아닙니다. 쿠키에 실려 오가는 값은 pk_sessions 테이블의 행 하나를 가리키는 무작위 id일 뿐이고, 값 자체에는 아무 정보도 담겨 있지 않습니다. 그래서 로그아웃은 그 행을 지우는 것이고, 지우는 순간 같은 값으로는 아무것도 열리지 않습니다.

	기록된 값
로그아웃 전	GET /api/private-notes (Cookie: pk_session=.....) → 200, 내 자료 3건
로그아웃 후	GET /api/private-notes (같은 값) → 401 {"error": "로그인이 필요합니다."}
설명	로그아웃이 세션 행을 지우므로, 그 값은 그 즉시 아무 의미가 없다.

PASS · 검사 25 (T08-C33)

등록·로그인·로그아웃·비공개 자료 조회가 소스 어디를 지나는가 (T08-C49)

흐름	지나는 자리	하는 일
등록	app.js registerPasskey → api/register/options.js → 기기 → api/register/verify.js → lib/store.js	질문 발급·보관 → 기기가 키 쌍 생성 → 서명·origin 검증 → 공개키 저장 → 세션 발급
로그인	app.js loginWithPasskey → api/login/options.js → 기기 → api/login/verify.js	새 질문 발급 → 기기가 개인키로 서명 → 저장된 공개키로 확인 → 질문 소진 → 세션 발급
응답(누구인지)	lib/session.js currentUser / requireUser ← api/me.js	쿠키의 세션 id로 pk_sessions 조회. 없으면 401
비공개 조회	api/private-notes/index.js api/private-notes/[id].js	requireUser 통과 뒤, 세션이 가리키는 계정의 자료만 읽고 씬. 남의 것 이면 404

④ 안 열리는 것을 확인한 기록

확인 네 가지를 성공한 요청과 거절된 요청을 나란히 둡니다. "막았습니다"라는 문장이 아니라 막히는 장면이 근거입니다.

확인 1. 로그인하지 않고 비공개 자료를 열어 본다

	기록된 값
요청 1	GET https://aleph-passkey.vercel.app/api/private-notes (쿠키 없음)
응답 1	401 {"error":"로그인이 필요합니다."}
요청 2	GET https://aleph-passkey.vercel.app/api/private-notes/00000000-... (쿠키 없음)
응답 2	401 {"error":"로그인이 필요합니다."}
설명	화면에서 숨긴 것이 아니라 서버가 401로 끊는다. 몸통에 비공개 내용이 한 글자도 없다.

PASS · 검사 6 (T08-C16) PASS · 검사 7 (T08-C17)

	기록된 값
길이	29763바이트
비공개 문구 포함 여부	없음 – 비공개 내용은 인증을 통과한 뒤 app.js가 받아 와 채운다
설명	CSS로 숨기는 방식이었다면 소스에는 그대로 남아 있었을 것이다.

PASS · 검사 5 (T08-C18)

확인 2. 남의 패스키로 다른 계정의 자료를 열어 본다

	기록된 값
B → 자기 자료	GET /api/private-notes/b053e627-77b4-4e88-bc6c-7f4d12f3e1cd → 200 "[촬영] B만의 메모"
B → A의 자료	GET /api/private-notes/fac49231-a3ec-4851-a50d-42befba9124e → 404 "그런 자료가 없습니다."
A → 자기 자료	GET /api/private-notes/fac49231-a3ec-4851-a50d-42befba9124e → 200 "[촬영] A만의 메모"
A → B의 자료	GET /api/private-notes/b053e627-77b4-4e88-bc6c-7f4d12f3e1cd → 404 "그런 자료가 없습니다."
왜 403이 아니라 404인가	403은 '있지만 권한이 없다'는 뜻이라, 그 id가 존재한다는 사실 자체를 알려주게 된다. 그래서 없는 것과 똑같이 답한다.

PASS · 검사 31 (T08-C37) PASS · 검사 32 (T08-C38)

	기록된 값
거절 전 A의 건수	4건

	기록된 값
거절 후 A의 건수	4건
설명	거절이 상대방 자료를 건드리지도, 새로 만들지도 않았다.

PASS · 검사 33 (T08-C39)

	기록된 값
요청	POST /api/private-notes 본문: { "title": "...", "user_id": "(A의 계정 id)" } ← B의 세션으로 보냄
저장결과	201 – 저장된 주인은 B(요청을 보낸 사람)
A가 그 자료를 볼 수 있는가	GET /api/private-notes/ca122779-044c-4666-b133-0523411a9d23 → 404 (못 본다)
B의 목록에는	있다
이 거절을 만드는 곳	api/private-notes/index.js – createNote에 넘기는 userId가 요청 본문이 아니라 세션(requireUser)에서 나온다. 클라이언트가 주인을 고를 수 있는 경로가 코드에 아예 없다.

PASS · 검사 34 (T08-C40)

확인 3. 이미 쓴 질문을 다시 보낸다

	기록된 값
첫 번째 요청	POST /api/login/verify (challengeId 5ff001a5-7ab2-4233-9ad2-9abfc66a2bc9) → 200
두 번째 요청	POST /api/login/verify (같은 challengeId, 같은 서명) → 400 "이미 사용된 확인 질문입니다."
설명	질문이 쓰이는 순간 used_at이 찍혀, 같은 서명을 재전송해도 통하지 않는다.

PASS · 검사 22 (T08-C31)

	기록된 값
성공 — 요청	POST /api/login/verify (기기가 개인키로 서명한 응답 그대로)
성공 — 응답	200 {"ok":true,"deviceName":["촬영] 노트북 지문"]}
실패 — 요청	POST /api/login/verify (같은 주소·같은 방식, 서명만 한 글자 다름)
실패 — 응답	401 {"error":"서명을 확인하지 못했습니다."}
설명	서버가 저장해 둔 공개키로 서명을 확인한 뒤에만 통과시킨다.

PASS · 검사 20 (T08-C29) PASS · 검사 21 (T08-C30)**확인 4. 패스키를 지운 뒤 그 패스키로 들어가 본다**

	기록된 값
지운것	"[촬영] 노트북 지문" (재확인 요구됨: true · 삭제 후 남은 패스키 1개)
지운 기기로 로그인	401 "등록되지 않은 패스키입니다."
남은 기기로 로그인	200 성공 - "[촬영] 휴대폰 지문"
설명	패스키를 두 개 등록해 둔 덕분에, 기기 하나를 잃어도 계정을 잃지 않는다.

PASS · 검사 28 (T08-C44) PASS · 검사 29 (T08-C45)

05 — PRIVATE

🔒 나만 보는 자리

열림

위쪽 소개는 누구나 볼 수 있습니다. 여기서부터는 패스키를 등록한 본인만 열 수 있습니다. 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 공개키만 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

프로젝트 메모

준비 중인 프로젝트 메모

학습 기록을 주 단위로 묶어 보여주는 작은 도구를 구상 중. 지금은 화면 스케치만 있고 아직 코드는 없다. 공개하기엔 이르다.

지원 목록

지원하려는 곳 목록

가상의 후보 세 곳 — 세로소프트(백엔드), 아르카 데이터터(플랫폼), 미르랩(사내 도구). 이력서 초안은 다음 달까지 정리 예정.

회고

스스로 쓰는 회고

이번 주에 배운 것: 화면에서 숨기는 것과 서버가 거절하는 것은 전혀 다르다는 것. 다음 주에 고칠 것: 막히는 장면을 먼저 만들고 나서 기능을 붙이기.

한 줄 더 남기기

회고

제목

내용 — 실제 개인정보는 넣지 않습니다

저장

등록된 패스키 2개

패스키는 기기에 매어 있습니다. 기기를 잃어버렸을 때를 대비해 두 개 이상 등록해 두세요 — 하나를 지워도 남은 것으로 들어올 수 있습니다.

[촬영] 노트북 지문 지금 이 기기

2026. 9. 8. 등록 · YAb89YWejdWm...

▶ 서버가 가진 값 보기

지우기

[촬영] 휴대폰 지문

2026. 9. 8. 등록 · zOI8QJkLWhuW...

▶ 서버가 가진 값 보기

지우기

패스키 추가

로그아웃

한 계정에 패스키 두 개 — 기기를 잃어버렸을 때를 위해

	기록된 값
화면 경고	△ 마지막 패스키입니다. 지우면 이 계정에 다시 들어올 방법이 없습니다 — 비밀번호도 이메일도 없어서 되돌릴 수 없습니다.
결과	남은 패스키 0개, 계정 접근 불가 = true

	기록된 값
다시 들어가려는 시도	401 "등록되지 않은 패스키입니다."
왜 막지 않았나	막을 수도 있었지만 막지 않았다. 되살릴 수단(이메일·비밀번호)을 두지 않기로 한 이상, '못 지우게 하는 것'은 문제를 미루는 것이지 푸는 것이 아니다. 대신 지우기 전에 화면에서 분명히 경고한다.

PASS · 검사 35 (T08-C46)

 나만 보는 자리

열림

위쪽 소개는 누구나 볼 수 있습니다. 여기서부터는 패스키를 등록한 본인만 열 수 있습니다. 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 공개키만 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

프로젝트 메모

준비 중인 프로젝트 메모

학습 기록을 주 단위로 묶어 보여주는 작은 도구를 구상 중. 지금은 화면 스케치만 있고 아직 코드는 없다. 공개하기엔 이르다.

지원 목록

지원하려는 곳 목록

가상의 후보 세 곳 — 세로소프트(백엔드), 아르카 데이터(플랫폼), 미르랩(사내 도구). 이력서 초안은 다음 달까지 정리 예정.

회고

스스로 쓰는 회고

이번 주에 배운 것: 화면에서 숨기는 것과 서버가 거절하는 것은 전혀 다르다는 것. 다음 주에 고칠 것: 막히는 장면을 먼저 만들고 나서 기능을 붙이기.

한 줄 더 남기기

회고

제목

내용 — 실제 개인정보는 넣지 않습니다

저장

등록된 패스키 1개

패스키는 기기에 매어 있습니다. 기기를 잃어버렸을 때를 대비해 두 개 이상 등록해 두세요 — 하나를 지워도 남은 것으로 들어올 수 있습니다.

⚠ 패스키가 하나뿐입니다. 이 기기를 잃어버리면 이 계정에 다시 들어올 방법이 없습니다 — 비밀번호도 이메일도 없어서 되살려 드릴 수단이 없습니다. 다른 기기에서 패스키를 하나 더 등록해 두세요.

[촬영] 휴대폰 지문

지금 이 기기

2026. 9. 8. 등록 · zO18QJkLWhuW...

▶ 서버가 가진 값 보기

⚠ 마지막 패스키입니다. 지우면 이 계정에 다시 들어올 방법이 없습니다 — 비밀번호도 이메일도 없어서 되돌릴 수 없습니다.

네, 지웁니다 — 되돌릴 수 없습니다

취소

패스키 추가

로그아웃

마지막 패스키를 지우기 전, 화면이 되돌릴 수 없음을 먼저 말한다

⑤ AI와 나

AI에게 맡긴 일, 내가 직접 판단한 일, AI 제안을 따르지 않은 일을 나눠 적습니다 (T08-C53).

AI에게 맡긴 일

WebAuthn 등록·로그인 흐름의 구현(서버 API와 브라우저 연결 코드), 저장소 스키마, 그리고 가상 authenticator를 쓰는 검사·촬영 도구 작성을 맡겼습니다. 표준 규격을 정확히 옮기는 일과, 손으로는 재현하기 번거로운 “기기를 바꿔 끼우는” 시나리오를 자동화하는 일이었습니다.

내가 직접 판단한 일

어떤 인증 수단을 쓸지 — 지문·Face ID 같은 기기 내장 방식을 기본으로 하고, 기기 분실 대비 두 번째 패스키는 다른 기기나 USB 보안 키로 두기로 정했습니다. 무엇을 공개하고 무엇을 비공개로 둘지(프로젝트 메모·지원 목록·회고), 그리고 비공개 자리에 **실제 개인정보를 넣지 않고 전부 지어낸 내용만 쓰기로** 한 것도 제 판단입니다.

AI 제안을 따르지 않은 일

처음 설계안은 브라우저 쪽도 @simplewebauthn/browser를 외부 CDN에서 불러다 쓰자는 것이었습니다. 따르지 않았습니다 — 브라우저 쪽이 하는 일은 자료 형식 변환 40줄이 전부인데, 그것 때문에 페이지가 외부 CDN에 의존하게 되는 것이 맞지 않다고 봤습니다. 정작 중요한 **서명 검증**은 그대로 표준 라이브러리에 맡겼습니다.

⑥ 아직 못 막은 것

먼저, 이 항목에 적었다가 실제로 막은 것들입니다. ⑥은 비워 두지 않는 자리이지만, 적어 놓고 그대로 두는 자리도 아닙니다. 앞선 제출 뒤에 아래 셋을 코드로 고쳤고, 고친 것을 말이 아니라 검사로 남겼습니다.

- **잠금이 풀린 기기** — 등록·로그인 모두 `userVerification: "required"`로 올렸고, 서버도 응답의 UV 플래그를 `requireUserVerification: true`로 다시 보냅니다. **PASS · 검사 36 (⑥-1)**
- **로그인만으로 패스키 삭제** — 되돌릴 수 없는 동작에는 5분 이내의 패스키 재확인을 요구합니다. **PASS · 검사 37 (⑥-2) PASS · 검사 38 (⑥-2)**
- **다 쓴 질문이 쌓이는 것** — 질문을 새로 만들 때 만료된 지 한 시간이 지난 줄을 함께 지웁니다.

그리고 제출 뒤에 실제 사용 중 발견해서 고친 것이 하나 더 있습니다 — 이게 가장 중요합니다.

패스키를 지우는 사이에 다른 패스키까지 지워져 계정이 통째로 사라졌습니다. 삭제 한 번은 첫 DELETE → 403 → 패스키 재확인 → 재시도로 몇 초가 걸립니다. 그동안 화면이 그대로 살아 있어서, "안 눌렀나?" 하고 다른 줄의 지우기를 누르면 두 요청이 나란히 나가 패스키가 둘 다 지워졌습니다. 이메일도 비밀번호도 없는 계정이라 그건 곧 계정 소멸입니다. 실제로 재현했을 때 `/api/me`가 401, 남은 패스키 0개가 됐습니다.

통과 기준 검사(28-29-35)는 이걸 잡지 못했습니다 — 삭제를 순차로 한 번씩만 시험했기 때문입니다. 사람이 조금하게 두 번 누르는 상황이 검사에 없었습니다.

고친 방법 — 확인을 누르는 즉시 목록 전체를 잠그고(버튼 disabled + 처리기 재진입 차단), 무엇이 도는 중인지 화면에 바로 적습니다. 그리고 그 상황을 재현하는 검사를 새로 넣었습니다. **PASS · 검사 40 (⑥-2) PASS · 검사 41 (⑥-2)**

같이 넣은 것 — 목록의 각 줄에 "지금 이 기기" 표시를 붙여, 지금 로그인에 쓰고 있는 패스키가 어느 것인지 눈으로 구분되게 했습니다(세션에 `credential_id`를 남깁니다). **PASS · 검사 42 (보강)**

이제 남은 것들입니다. 다 막았다고 적지 않겠습니다.

1. **아무나 계정을 얼마든지 만들 수 있습니다.** `/api/register/options`에 요청 수 제한이 없습니다. 비밀번호가 없으니 "비밀번호 무차별 대입"은 애초에 성립하지 않지만, 등록 자체를 자동으로 반복해 DB를 부풀리는 것은 지금 막지 못합니다.
2. **비공개 자료는 서버에 그냥 저장됩니다.** 자료 자체를 암호화하지 않았기 때문에, 서버의 `DATABASE_URL`이 새면 전부 읽힙니다. 패스키가 막는 것은 "바깥에서 들어오는 길"이지 "서버 안에서 보는 눈"이 아닙니다.
3. **계정을 되살릴 방법이 전혀 없습니다.** 기기를 전부 잃거나 마지막 패스키를 지우면 그 계정은 영원히 닫힙니다(검사 35). 일부러 그렇게 두었고 화면에서 경고도 하지만, 실제 서비스라면 복구 코드 같은 예비 수단이 필요합니다.
4. **실제 기기 두 대로 카드 4를 시연하지 못했습니다.** 아이폰 Face ID로 직접 등록해 저장 위치(iCloud 키체인)까지는 확인했지만, 두 번째로 쓰려던 윈도우 PC가 공용 PC라 Windows Hello PIN을 새로 만들 수 없었고 블루투스도 켜지지 않아 QR 로그인도 안 됐습니다. 공용 PC에 새 자격 증명을 만드는 것은 부적절해 시도를 접었습니다. **보안 구멍이 아니라 이번 확인의 환경 제약**이고, 메커니즘 자체는 가상 authenticator 검사(26~29-35)가 공개 주소에서 그대로 확인했습니다.
5. **동시 조작 경로를 전부 흘리는 못했습니다.** 위의 삭제 경험은 찾아서 막았지만, 그건 실제로 써 보다가 걸린 하나입니다. 같은 종류(느린 요청이 도는 동안 화면이 살아 있는 곳)의 다른 경로 — 등록과 삭제를 겹쳐 누르는 경우 같은 것 — 은 검사로 흘리지 않았습니다. 하나를 찾았다는 것은 같은 자리가 더 있을 수 있다는 뜻이라, 없다고 적지 않겠습니다.

카드 1 — 무엇을 잠갔는가

공개 영역은 과제 1에서 만든 소개 페이지 그대로입니다. 문장 하나 바꾸지 않았고, 검사 2가 원본 커밋(cb23773)에서 글자를 뽑아 지금 페이지와 한 줄씩 대조합니다. **PASS · 검사 2 (T08-C11)**

그 아래에  **나만 보는 자리**를 새로 만들었습니다. 배경색과 테두리를 달리해 어디까지가 공개이고 어디부터가 비공개인지 한눈에 보이게 했고, 지금 잠겨 있는지 열려 있는지도 배지로 밝힙니다. **PASS · 검사 3 (T08-C13) PASS · 검사 1 (T08-C10)**

JIN.

About

공개 범위

Strengths

Proof

 나만 보는 자리

GitHub ↗

IT PORTFOLIO

Hello, I'm JIN.

IT 분야를 공부하며 직접 부딪히고, 문제를 해결하며 배우고 있습니다.

INTRODUCTION

문제가 생기면 원인을 찾습니다.

ACTIVITY

Java · Spring Boot · Git · GitHub

PROOF

소셜 로그인 오류를 분석하고 해결한 경험

자세히 보기 ↓

05 — PRIVATE



나만 보는 자리

잠금 — 패스키가 있어야 열립니다

위쪽 소개는 누구나 볼 수 있습니다. **여기서부터는 패스키를 등록한 본인만** 열 수 있습니다. 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 **공개키만** 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

패스키로 들어가기

패스키 새로 등록하기

잠긴 상태 — 무엇이 있는지도 보이지 않는다

05 — PRIVATE

🔒 나만 보는 자리

열림

위쪽 소개는 누구나 볼 수 있습니다. **여기서부터는 패스키를 등록한 본인만** 열 수 있습니다. 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 **공개키만** 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

프로젝트 메모

준비 중인 프로젝트 메모

학습 기록을 주 단위로 묶어 보여주는 작은 도구를 구상 중. 지금은 화면 스케치만 있고 아직 코드는 없다. 공개하기엔 이르다.

지원 목록

지원하려는 곳 목록

가상의 후보 세 곳 — 세로소프트(백엔드), 아르카 데이터(플랫폼), 미르랩(사내 도구). 이력서 초안은 다음 달까지 정리 예정.

회고

스스로 쓰는 회고

이번 주에 배운 것: 화면에서 숨기는 것과 서버가 거절하는 것은 전혀 다르다는 것. 다음 주에 고칠 것: 막히는 장면을 먼저 만들고 나서 기능을 붙이기.

한 줄 더 남기기

회고

제목

내용 — 실제 개인정보는 넣지 않습니다

저장

등록된 패스키 1개

패스키는 기기에 매여 있습니다. 기기를 잃어버렸을 때를 대비해 **두 개 이상** 등록해 두세요 — 하나를 지워도 남은 것으로 들어올 수 있습니다.

⚠ 패스키가 하나뿐입니다. 이 기기를 잃어버리면 이 계정에 다시 들어올 방법이 없습니다 — 비밀번호도 이메일도 없어서 되살려 드릴 수단이 없습니다. 다른 기기에서 **패스키를 하나 더 등록해** 두세요.

[촬영] 노트북 지문

2026. 9. 8. 등록 · YAb89YWejdWm...

▶ 서버가 가진 값 보기

지우기

패스키 추가

로그아웃

열린 상태 — 프로젝트 메모·지원 목록·회고 세 항목 (전부 지어낸 내용)

PASS · 검사 4 (T08-C15) PASS · 검사 17 (T08-C14) PASS · 검사 9 (T08-C12)

비공개 내용이 **소스에 아예 없는** 이유 — 화면의 비공개 자리는 처음에 빈 칸으로만 그려집니다. 인증을 통과해야 app.js가 서버에서 내용을 받아 채웁니다. CSS로 숨겼다면 페이지 소스에는 그대로 남았을 것입니다.

카드 2 — 패스키를 등록한다

	기록된 값
첫 번째 요청	POST /api/register/options
첫 번째 challenge	JAk1SnLsNpQhxDGZqls719417VjVf2FIuSAWB1I16A
두 번째 challenge	Rh_4uFRgSwyUeZAfFI6KXppVoSrZX1ICAbdBKmeLdLA
같은가	다르다
보관	서버가 pk_challenges에 넣어 두고, 확인 단계에서 딱 한 번만 쓴 뒤 used_at을 찍는다

PASS · 검사 10 (T08-C19) PASS · 검사 11 (T08-C20)

05 — PRIVATE


나만 보는 자리

잠김 — 패스키가 있어야 열립니다

위쪽 소개는 누구나 볼 수 있습니다. 여기서부터는 패스키를 등록한 본인만 열 수 있습니다. 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 공개키만 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

패스키로 들어가기

패스키 새로 등록하기

이 패스키의 이름 — 나중에 목록에서 알아볼 이름입니다

등록하기

취소

패스키마다 사람이 알아볼 이름을 붙인다

PASS · 검사 16 (T08-C24)

	기록된 값
결과	기기 확인 단계에서 취소됨 (AbortError)
화면 안내	등록을 취소했습니다. 서버에는 계정도 패스키도 만들어지지 않았습니다.
서버에 저장된 것	없음 — 계정도 자격증명도 만들어지지 않는다. 남는 것은 2분 뒤 만료되는 질문 한 줄뿐이고, 그건 계정이 아니다.

PASS · 검사 12 (T08-C25)

카드 3 — 패스키로 들어간다

	기록된 값
첫 번째 challenge	38Wd60_3qKXH_dbfKkG479JPSpeXOWR_J351EmyqBfY
두 번째 challenge	EVBY3038ZJAB4msskFPwWvTowzxOE8sjsDi4_sNNmyc
같은가	다르다

PASS · 검사 18 (T08-C27) PASS · 검사 19 (T08-C28)

카드 4 — 기기를 잃어버렸을 때

패스키는 기기에 매여 있습니다. 하나뿐이면 그 기기를 잃는 순간 계정도 잃습니다. 그래서 **패스키를 두 개 등록해 두**고, 하나를 지운 뒤에도 남은 하나로 들어갈 수 있는지 실제로 확인했습니다.

PASS · 검사 26 (T08-C42) PASS · 검사 27 (T08-C43)

 나만 보는 자리

열림

위쪽 소개는 누구나 볼 수 있습니다. 여기서부터는 패스키를 등록한 본인만 열 수 있습니다. 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 공개키만 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

프로젝트 메모

준비 중인 프로젝트 메모

학습 기록을 주 단위로 묶어 보여주는 작은 도구를 구상 중. 지금은 화면 스케치만 있고 아직 코드는 없다. 공개하기엔 이르다.

지원 목록

지원하려는 곳 목록

가상의 후보 세 곳 — 세로소프트(백엔드), 아르카 데이터터(플랫폼), 미르랩(사내 도구). 이력서 초안은 다음 달까지 정리 예정.

회고

스스로 쓰는 회고

이번 주에 배운 것: 화면에서 숨기는 것과 서버가 거절하는 것은 전혀 다르다는 것. 다음 주에 고칠 것: 막히는 장면을 먼저 만들고 나서 기능을 붙이기.

한 줄 더 남기기

회고

제목

내용 — 실제 개인정보는 넣지 않습니다

저장

등록된 패스키 2개

패스키는 기기에 매어 있습니다. 기기를 잃어버렸을 때를 대비해 두 개 이상 등록해 두세요 — 하나를 지워도 남은 것으로 들어올 수 있습니다.

[촬영] 노트북 지문

2026. 9. 8. 등록 · YAb89YWejdWm...

▶ 서버가 가진 값 보기

지문 패스키로는 더 이상 들어올 수 없습니다. 남은 패스키로는 그대로 들어올 수 있습니다.

네, 지웁니다 — 되돌릴 수 없습니다

취소

[촬영] 휴대폰 지문

2026. 9. 8. 등록 · zOI8QJkLWhuW...

▶ 서버가 가진 값 보기

 지우기

패스키 추가

로그아웃

지우기는 두 단계 — 무엇이 사라지는지 먼저 말한다

05 — PRIVATE

🔒 나만 보는 자리

잠김 — 패스키가 있어야 열립니다

위쪽 소개는 누구나 볼 수 있습니다. 여기서부터는 **패스키를 등록한 본인만** 열 수 있습니다. 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 **공개키만** 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

패스키로 들어가기

패스키 새로 등록하기

이 패스키의 이름 — 나중에 목록에서 알아볼 이름입니다

등록하기

취소

마지막 패스키를 지운 뒤 — 다시 잠기고, 되돌릴 수 없다고 알린다

실제 기기 두 대로는 시연하지 못한 부분 — 위 자동 검사(26~29:35)는 가상 authenticator 두 개로 "패스키 하나를 잃고 다른 하나로 들어가는" 과정을 재현한 것이다. 실제 기기 두 대(아이폰 Face ID + 다른 기기)로 같은 것을 해 보려 했으나, 두 번째 기기로 쓸 수 있었던 윈도우 PC가 **공용 PC라 로그인 PIN을 새로 만들 수 없었고**(Windows Hello가 아예 없어 물리 보안 키를 요구함), **블루투스도 켜지지 않아** 휴대폰을 거쳐 로그인하는 방법(QR)도 쓸 수 없었다. 공용 PC에 새 자격 증명을 만드는 것 자체가 부적절한 일이라 시도를 접었다. 그래서 이 흐름은 실제 기기 두 대가 아니라 자동화된 가상 authenticator 검사로만 확인됐다 — 그 검사는 공개 배포 주소(a1eph-passkey.vercel.app)를 대상으로 돌았다.

모바일

05 — PRIVATE



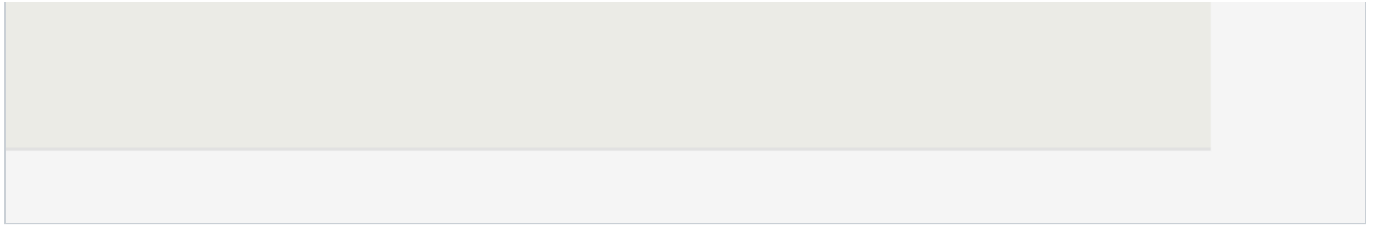
나만 보는 자리

잠김 — 패스키가 있어야 열립니다

위쪽 소개는 누구나 볼 수 있습니다. **여기서부터는 패스키를 등록한 본인만 열 수 있습니다.** 이 사이트는 비밀번호를 아예 만들지 않습니다 — 등록할 때 기기가 열쇠 한 쌍을 만들어 **공개키만** 서버로 보내고, 개인키는 기기 밖으로 나가지 않습니다.

패스키로 들어가기

패스키 새로 등록하기



가로 375px에서도 공개·비공개 경계와 잠금 상태가 그대로 보인다

검사 42개

사람 눈이 아니라 명령 하나가 판정합니다. 패스키는 원래 지문이나 보안 키가 있어야 시험할 수 있지만, 크롬 개발자 프로토콜의 **가상 authenticator**를 붙이면 실제 기기와 똑같이 키 쌍을 만들고 서명하게 할 수 있어 등록·로그인·기기 분실·계정 격리를 전부 자동으로 재현했습니다.

검사 대상: <https://aleph-passkey.vercel.app> · 기록: [portfolio-passkey-2026-09-08-16-51.json](#)

#	코드	카드	검사	결과
1	T08-C10	카드 5	첫 화면이 공개 소개 페이지이고, 아무것도 등록하지 않아도 열린다	PASS
2	T08-C11	카드 5	과제 1의 공개 내용이 한 줄도 빠짐없이 그대로 남아 있다	PASS
3	T08-C13	카드 1	공개 영역과 비공개 영역이 화면에서 구분되어 보인다	PASS
4	T08-C15	카드 1	패스키로 들어가지 않은 상태에서는 비공개 내용이 화면에 없다	PASS
5	T08-C18	카드 1	로그인하지 않은 채 받은 페이지 소스 어디에도 비공개 내용이 없다	PASS
6	T08-C16	카드 1	비공개 자료를 서버에 직접 요청하면 거절된다	PASS
7	T08-C17	카드 1	그 거절이 401 또는 403으로 이루어진다	PASS
8	T08-C35	카드 3	제출물 어디에도 비밀번호를 입력하는 칸이 없다	PASS
9	T08-C12	카드 5	제출물에 실제 연락처·신분증 번호 같은 개인정보가 없다	PASS
10	T08-C19	카드 2	서버가 등록용 질문을 만들어 보내고, 확인할 때까지 보관한다	PASS
11	T08-C20	카드 2	등록 요청마다 질문 값이 서로 다르다	PASS
12	T08-C25	카드 2	등록을 중간에 취소하면 계정도 패스키도 만들어지지 않는다	PASS
13	T08-C21	카드 2	등록이 끝나면 서버에 공개키가 저장된다	PASS
14	T08-C22	카드 2	저장된 값이 공개키다 — 비밀번호가 아니다	PASS
15	T08-C23	카드 2	등록 요청 본문에 개인키가 실려 있지 않다	PASS

#	코드	카드	검사	결과
16	T08-C24	카드 2	등록한 패스키에 사람이 알아볼 수 있는 이름이 붙는다	PASS
17	T08-C14	카드 1	비공개 영역에 들어간 항목이 세 개 이상이다	PASS
18	T08-C27	카드 3	로그인할 때도 서버가 매번 새 질문을 만들어 보낸다	PASS
19	T08-C28	카드 3	로그인 요청마다 질문 값이 서로 다르다	PASS
20	T08-C29	카드 3	서버가 저장해 둔 공개키로 서명을 확인한 뒤에만 통과시킨다	PASS
21	T08-C30	카드 3	서명이 한 글자만 달라도 거절된다 (성공 요청과 나란히)	PASS
22	T08-C31	카드 3	이미 한 번 쓴 질문으로 다시 로그인하려 하면 거절된다	PASS
23	T08-C32	카드 3	로그인 뒤 사람을 알아보는 것은 세션이다 (서버가 들고 있는 값)	PASS
24	T08-C34	카드 3	세션 값이 페이지의 자바스크립트에 노출되지 않는다	PASS
25	T08-C33	카드 3	로그아웃한 뒤 같은 세션 값으로 다시 요청하면 거절된다	PASS
26	T08-C42	카드 4	한 계정에 패스키가 두 개 등록되어 있다	PASS
27	T08-C43	카드 4	패스키 목록에 각각의 이름과 등록한 날짜가 보인다	PASS
28	T08-C44	카드 4	패스키 하나를 지운 뒤 남은 하나로 들어갈 수 있다	PASS
29	T08-C45	카드 4	지운 패스키로는 더 이상 들어갈 수 없다	PASS
30	T08-C36	카드 5	패스키를 등록한 계정이 두 개이고, 각각 서로 다른 비공개 내용이 있다	PASS
31	T08-C37	카드 5	한쪽 패스키로 다른 쪽 비공개 자료를 읽으려는 요청이 거절된다	PASS
32	T08-C38	카드 5	반대 방향(다른 쪽에서 첫 쪽으로)의 요청도 똑같이 거절된다	PASS
33	T08-C39	카드 5	거절 앞뒤로 반대편의 자료 건수가 같다	PASS
34	T08-C40	카드 5	요청 본문에 남의 계정 id를 적어 보내도 내 자료로만 저장된다	PASS

#	코드	카드	검사	결과
35	T08-C46	카드 4	마지막 패스키를 지우면 그 계정은 더 이상 열 수 없다	PASS
36	㉔-1	보강	등록·로그인 모두 지문·얼굴·PIN 확인을 반드시 거치게 되어 있다	PASS
37	㉔-2	보강	로그인만 되어 있다고 패스키를 지울 수는 없다 (재확인 요구)	PASS
38	㉔-2	보강	패스키로 다시 확인하면 그때 지워진다	PASS
39	㉔-4	보강	패스키가 하나뿐이면 화면이 그 위험을 먼저 알린다	PASS
40	㉔-2	보강	삭제가 도는 동안 다른 패스키의 지우기 버튼이 잠긴다	PASS
41	㉔-2	보강	삭제 중 버튼을 마구 눌러도 패스키가 하나만 지워진다 (계정이 사라지지 않는다)	PASS
42	보강	보강	목록에서 지금 로그인에 쓴 패스키가 어느 것인지 보인다	PASS